

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 8.1 Support Vector Machine Classification
5  Machine Learning for Engineering Problem Solving
6  @author: Austin R.J. Downey
7  """
8
9  import IPython as IP
10 IP.get_ipython().run_line_magic('reset', '-sf')
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import sklearn as sk
15 from sklearn import datasets
16 from sklearn import svm
17 from sklearn import pipeline
18
19 cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
20 plt.close('all')
21
22
23 # This code will build and train a support vector machine classifier with soft
24 # for the iris flower data set.
25
26 %% Load your data
27
28 # We will use the Iris data set. This dataset was created by biologist Ronald
29 # Fisher in his 1936 paper "The use of multiple measurements in taxonomic
30 # problems" as an example of linear discriminant analysis
31 iris = sk.datasets.load_iris()
32
33 # for simplicity, extract some of the data sets
34 X = iris['data'] # this contains the length of the petals and sepals
35 Y = iris['target'] # contains what type of flower it is
36 Y_names = iris['target_names'] # contains the name that aligns with the type of the
37 # flower
38 feature_names = iris['feature_names'] # the names of the features
39
40 # plot the Sepal data
41 plt.figure(figsize=(6.5,3))
42 plt.subplot(121)
43 plt.grid(True)
44 plt.scatter(X[Y==0,0],X[Y==0,1],marker='o',zorder=10)
45 plt.scatter(X[Y==1,0],X[Y==1,1],marker='s',zorder=10)
46 plt.scatter(X[Y==2,0],X[Y==2,1],marker='d',zorder=10)
47 plt.xlabel(feature_names[0])
48 plt.ylabel(feature_names[1])
49
50 plt.subplot(122)
51 plt.grid(True)
52 plt.scatter(X[Y==0,2],X[Y==0,3],marker='o',label=Y_names[0],zorder=10)
53 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',label=Y_names[1],zorder=10)
54 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',label=Y_names[2],zorder=10)
55 plt.xlabel(feature_names[2])
56 plt.ylabel(feature_names[3])
57 plt.legend(framealpha=1)
58 plt.tight_layout()
59
60 %% Extract just the petal space of the code
61
62 X_petal = X[50:, (2, 3)] # petal length, petal width
63 y_petal = Y[50:] == 2
64
65 %% Build and train the SVM classifier
66
67 # build handles to regularize the model data and a Linear Support Vector Classification.

```

```

67 scaler = sk.preprocessing.StandardScaler()
68 svm_clf = sk.svm.LinearSVC(C=1000000,max_iter=10000)
69
70 # build the model pipeline of regularization and a Linear Support Vector Classification.
71 scaled_svm_clf = sk.pipeline.Pipeline([
72     ("scaler", scaler),
73     ("linear_svc", svm_clf),
74 ])
75
76 # train the data
77 scaled_svm_clf.fit(X_petal, y_petal)
78
79
80 %% Build and plot the decision boundary along with the curbs
81
82 # Convert to unscaled parameters as the SVM is solved in a scaled space.
83 w = svm_clf.coef_[0] / scaler.scale_
84 b = svm_clf.decision_function([-scaler.mean_ / scaler.scale_])
85
86 # At the decision boundary,  $w_0 \cdot x_0 + w_1 \cdot x_1 + b = 0$ 
87 #  $\Rightarrow x_1 = -w_0/w_1 \cdot x_0 - b/w_1$ 
88 x0 = np.linspace(4, 5.9, 200)
89 decision_boundary = -w[0]/w[1] * x0 - b/w[1]
90
91 margin = 1/w[1]
92 curbs_up = decision_boundary + margin
93 curbs_down = decision_boundary - margin
94
95 %% Plot the data and the classifier
96
97 plt.figure()
98 plt.grid(True)
99 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',label=Y_names[1],zorder=10)
100 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',label=Y_names[2],zorder=10)
101 plt.xlabel(feature_names[2])
102 plt.ylabel(feature_names[3])
103 plt.legend(framealpha=1)
104 plt.tight_layout()
105
106 # plot the decision boundy and margins
107 plt.plot(x0, decision_boundary, "k-", linewidth=2)
108 plt.plot(x0, curbs_up, "k--", linewidth=2)
109 plt.plot(x0, curbs_down, "k--", linewidth=2)
110
111
112 %% Find the misclassified instances and add a circle to mark them
113
114 # Find support vectors (LinearSVC does not do this automatically) and add them
115 # to the SVM handle
116 t = y_petal * 2 - 1 # convert 0 and 1 to -1 and 1
117 support_vectors_idx = (t * (X_petal.dot(w) + b) < 1) # find the locations
118 # of the miss classified data points that fall withing the vectors
119 svcs = X_petal[support_vectors_idx]
120
121 plt.scatter(svs[:, 0], svcs[:, 1], s=180,marker='o', facecolors='none',edgecolors='k')
122
123 %% compute the confusion matirx and F1 score
124
125 y_predicted = scaled_svm_clf.predict(X_petal)
126 confusion_matrix = sk.metrics.confusion_matrix(y_predicted, y_petal)
127 f1_score = sk.metrics.f1_score(y_predicted, y_petal)
128
129 print(f1_score)

```